

25PC2RA202: Sensors and Actuators Laboratory (S&A Lab)

Experiment 1: Odometry and Kinematics

Prof. A. Adithya Kashyap, Prof. Harshvardhan Uppaluru

July 13/16, 2026

Student Name: _____

Number: _____

1. Introduction

In this experiment, you will understand and process **quadrature encoder** data from a live sensor to estimate the displacement and velocity of a wheel and robotics platform. You should be working in teams of 4.

2. Hardware

- Arduino R4 WiFi Board
- USB C-to-C cable
- Pro Range Digital Optical Rotary Encoder



3. Wiring

Wire	Signal	Arduino R4 Pin
Red	Supply Voltage	5V
Black	Ground	GND
Green	Phase A Output	D2 (Interrupt)
White	Phase B Output	D3
Shield/Bare Wire	Cable Shield	GND

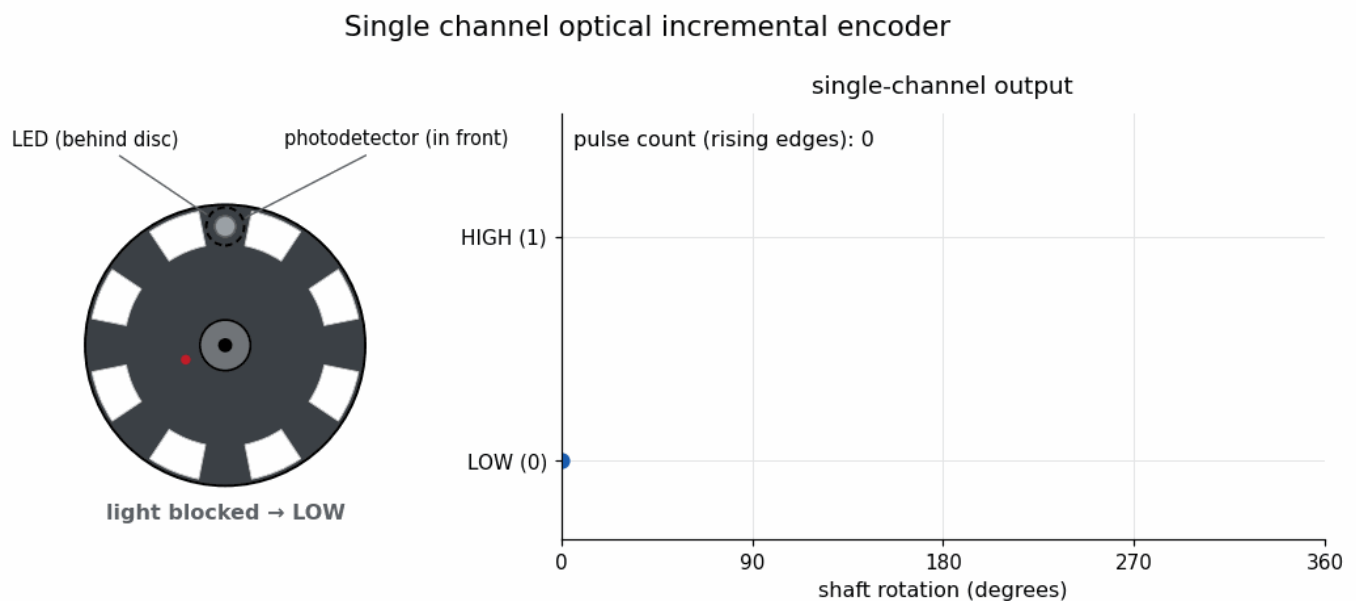
Caution: Do not connect the encoder supply to 3.3 V.

4. Theory

Wheel Encoders (sometimes also known as motor encoders) are devices used to measure the internal state of a mobile robot. Each wheel encoder is used to count the number of times the motor has rotated - This can be used to calculate the distance a robot has driven or turned. Wheel encoders are used to perform several mobile robot behaviors such as driving in a straight line or a curve. Advantages of wheel encoders include: Low cost, High quality and resolution.

Optical Incremental Encoder is a device that converts the rotational motion of a motor shaft into a train of digital pulses. It consists of three main parts: an LED light source, a coded disc mounted on the shaft with evenly spaced transparent and opaque segments, and a photodetector. As the shaft rotates, the segments alternately pass and block the light beam, so the photodetector outputs a square wave (1 (High) for detecting light, 0 (Low) otherwise). By counting these pulses, the angular displacement of the motor shaft relative to its starting position can be determined. The encoder measures a change in position, not absolute position - so every measurement must be referenced to a known zero. The resolution of an incremental encoder is specified in *Pulses Per Revolution (PPR)*.

Note: The encoder used in this experiment has a resolution of $N = 360$ PPR. This means that the encoder produces one pulse for every 1° of shaft rotation.



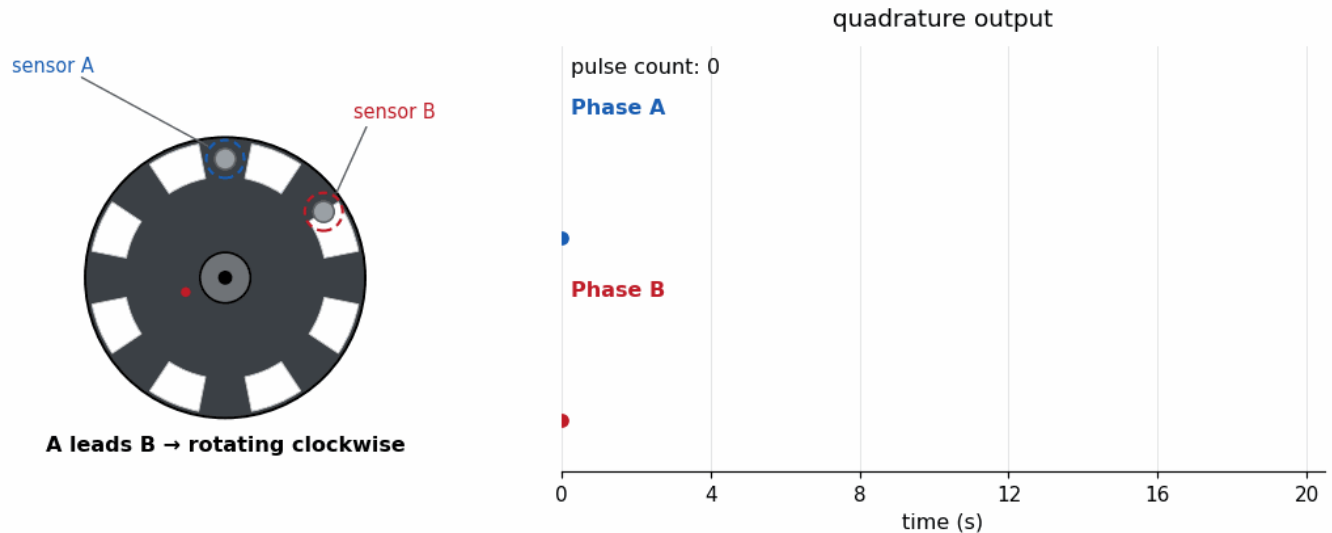
Quadrature Encoder solves an important disadvantage of single-channel encoder. It can measure displacement and speed, however it cannot detect the direction of rotation. Quadrature encoder solves this problem by producing two square wave outputs - Phase A and Phase B - generated by two photodetectors that are offset from each other by one quarter of a pulse-cycle (90°). The direction of rotation is determined from the phase relationship between the two channels:

- When the shaft rotates clockwise, Phase A leads Phase B.
- When the shaft rotates counter-clockwise, Phase B leads Phase A.

Think about this: What does this mean in practice?

$$\text{Measured Angle (in } ^\circ) = \text{Pulse Count} * 360^\circ * (1/N)$$

Two channel quadrature encoder



Calculating distance and velocity: When the quadrature encoder is coupled to a wheel, the recorded pulse count can be converted into the linear distance travelled by the wheel along the ground. One full revolution of the wheel covers a distance equal to its circumference ($2\pi R$). Since the encoder we use in our experiments produce 360 counts per revolution, the distance traveled is:

$$\text{Distance } (d) = \text{Pulse Count} * 2\pi R * (1/N)$$

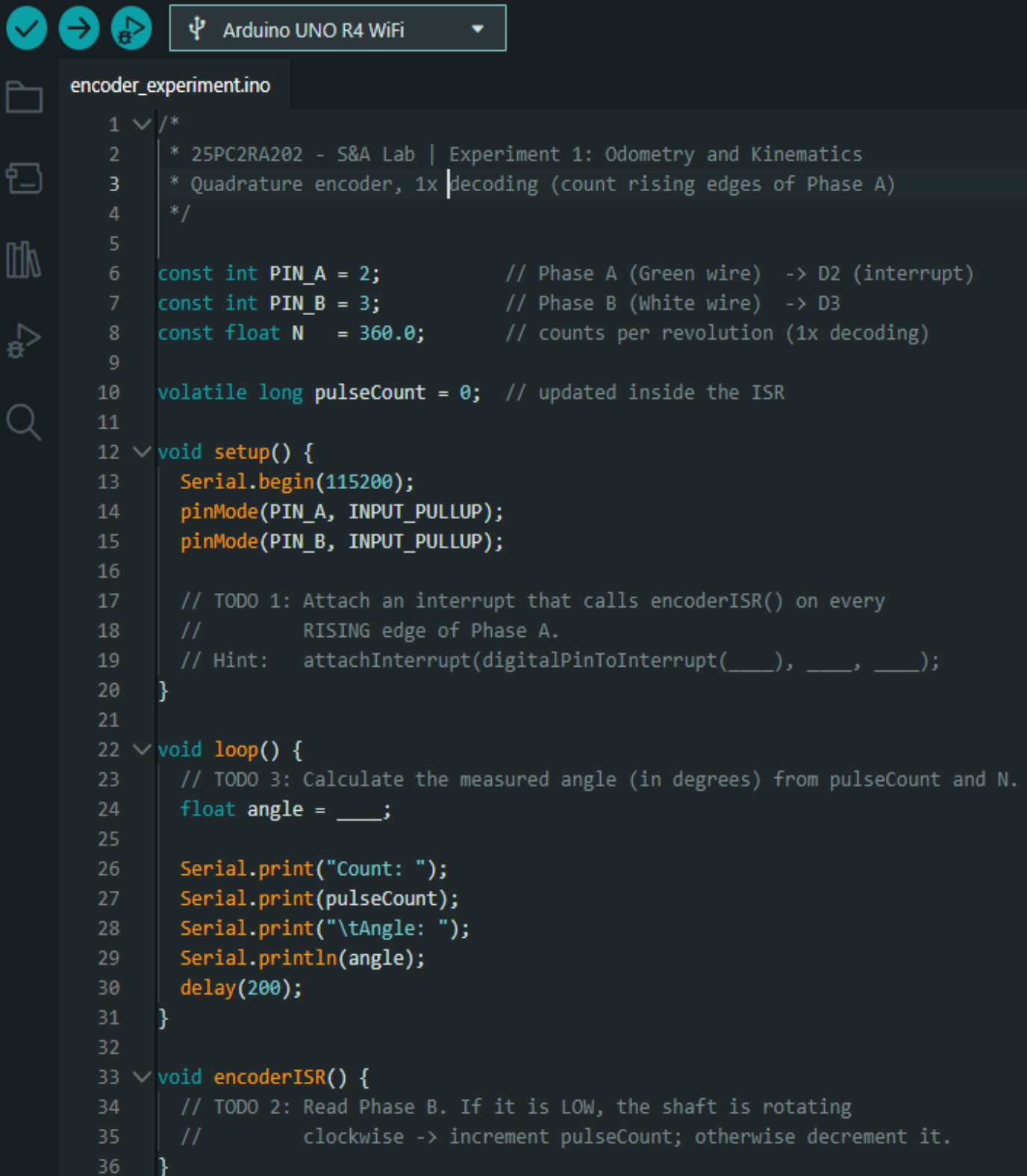
The average linear velocity of the wheel is the distance travelled divided by the elapsed time, and the angular velocity of the wheel follows from v and R .

$$v = d/\Delta t \text{ \& } \omega = v/R$$

Assume the following parameters to complete the remaining columns of the observation table:

- Wheel of radius (R) = 5 cm = 0.05 m.
- Elapsed time (Δt) = 2 seconds for each recorded rotation

5. Arduino Code



```
1  /*
2   * 25PC2RA202 - S&A Lab | Experiment 1: Odometry and Kinematics
3   * Quadrature encoder, 1x decoding (count rising edges of Phase A)
4   */
5
6   const int PIN_A = 2;           // Phase A (Green wire) -> D2 (interrupt)
7   const int PIN_B = 3;           // Phase B (White wire) -> D3
8   const float N = 360.0;         // counts per revolution (1x decoding)
9
10  volatile long pulseCount = 0; // updated inside the ISR
11
12  void setup() {
13      Serial.begin(115200);
14      pinMode(PIN_A, INPUT_PULLUP);
15      pinMode(PIN_B, INPUT_PULLUP);
16
17      // TODO 1: Attach an interrupt that calls encoderISR() on every
18      //          RISING edge of Phase A.
19      // Hint:   attachInterrupt(digitalPinToInterrupt(___), ___, ____);
20  }
21
22  void loop() {
23      // TODO 3: Calculate the measured angle (in degrees) from pulseCount and N.
24      float angle = ____;
25
26      Serial.print("Count: ");
27      Serial.print(pulseCount);
28      Serial.print("\tAngle: ");
29      Serial.println(angle);
30      delay(200);
31  }
32
33  void encoderISR() {
34      // TODO 2: Read Phase B. If it is LOW, the shaft is rotating
35      //          clockwise -> increment pulseCount; otherwise decrement it.
36  }
```

6. Procedure

- a. Mark a reference point on the encoder shaft with a marker.
- b. Complete the wiring as per Section with the Arduino R4 disconnected from the computer.
- c. Connect the Arduino R4 to the computer.
- d. Instructions on how to run arduino
 - i. Open Terminal (Ctrl + Alt + T)
 - ii. Run `./Applications/arduino-ide_2.3.10_Linux_64bit.ApplImage`
 - iii. Select Board - Arduino Uno R4 WiFi
- e. Complete the Arduino code - Learn about Arduino [here](#) (verify, upload and serial monitor). You will need to write how to trigger an interrupt and a simple interrupt service routine.
- f. Rotate the shaft (clockwise and counter-clockwise) slowly, observe the sign of the count and record the pulse count.
- g. Complete the observation tables. Before starting a new experiment, reset it to a default zero position.
- h. **Note: If you are saving any files, save it within the Arduino folder as .ino files ONLY!**

7. Observation Table:

Trial I - Clockwise (CW)

Rotation (Degrees)	Pulse Count	Distance Travelled (m)	Linear velocity (m/s)	Angular velocity (rad/s)
90				
180				
270				
360			n	

Trial II - Counter-Clockwise (CCW)

Rotation (Degrees)	Pulse Count	Distance Travelled (m)	Linear velocity (m/s)	Angular velocity (rad/s)
90				
180				
270				
360				

Trial III - Clockwise and Counter-Clockwise

Rotation (Degrees)	Cumulative Pulse Count	Δ Pulse Count	Distance Travelled (m)	Linear velocity (m/s)	Angular velocity (rad/s)
90 CCW					
180 CW					
270 CCW					
90 CW					
90 CW					

8. Evaluation Rubric

Assessment Component	Marks	Awarded Marks
Wiring	1	
Programming	2	
Observation Table	5	
Report Quality	1	
Attendance	1	
Total	10	

Signatures

Prof. A. Adithya Kashyap
Date:

Prof. Harshvardhan Uppaluru
Date: